

Lecture: Binary Search Algorithm — From Zero to Advanced

1. Introduction

Binary Search is one of the most efficient and fundamental algorithms in computer science.

It builds upon the idea of searching in **sorted data**, making it far faster than Linear Search.

Definition:

A Binary Search algorithm repeatedly divides a sorted collection in half, comparing the target with the middle element, and narrowing the search space until the target is found or the list is exhausted.

Real-life analogy:

Imagine searching for a word in a **dictionary**. You don't flip one page at a time. Instead, you open near the middle, decide if your word comes before or after, and keep halving the search space.

2. Key Characteristics

- **Type:** Divide-and-Conquer Search
- **Input Requirement:** Data must be **sorted** (ascending or descending).
- **Direction:** Jumps across halves instead of sequential scanning.
- **Time Complexity:** $O(\log n)$
- **Space Complexity:** $O(1)$ for iterative, $O(\log n)$ for recursive (due to call stack).

3. Step-by-Step Working

1. Start with $low = 0$ and $high = n-1$.
2. Find $mid = (low + high) // 2$.
3. Compare $arr[mid]$ with target:
 - If equal $\rightarrow$ return index.
 - If $target < arr[mid] \rightarrow$ search left half.
 - If $target > arr[mid] \rightarrow$ search right half.
4. Repeat until $low > high \rightarrow$ target not found.

4. Pseudocode

BinarySearch(arr, target):

 low = 0

 high = length(arr) - 1

 while low <= high:

 mid = (low + high) // 2

 if arr[mid] == target:

 return mid

 elif arr[mid] < target:

 low = mid + 1

 else:

 high = mid - 1

 return -1

5. Complexity Analysis

- **Best Case:** $O(1)$ → target is at the middle.
- **Worst Case:** $O(\log n)$ → search keeps halving until element not found.
- **Average Case:** $O(\log n)$.
- **Space Complexity:**
 - Iterative: $O(1)$.
 - Recursive: $O(\log n)$.

6. Handling Edge Cases

1. Empty list → return -1.
2. Single element list → check directly.
3. Multiple occurrences → return first/last index using modified binary search.
4. Data types → works with integers, floats, strings (if sorted).

- 5. Invalid input (unsorted list) → incorrect results.

7. Variations of Binary Search

- **Recursive Binary Search** → implemented with function calls.
- **First and Last Occurrence Search** → find boundaries in duplicates.
- **Binary Search on Answer (Search Space Reduction)** → used in optimization problems.
- **Exponential Search** → combines with Binary Search for unbounded arrays.

8. When to Use Binary Search

Best suited for:

- Large datasets.
- Sorted arrays or databases.
- Frequent searches with minimal updates.

Avoid if:

- Dataset is **unsorted** (sorting cost is $O(n \log n)$, making it inefficient).
- Small datasets where Linear Search may be simpler.

9. Advanced Usage

- Searching in **rotated sorted arrays** (Modified Binary Search).
- Searching in **2D matrices**.
- Used in algorithms like **Divide & Conquer**, **Binary Search Trees (BSTs)**, and **searching in dictionaries or indexes**.

10. Real-World Applications

- Searching for a contact in a **phonebook** (sorted by name).
- Database indexing.
- Search engines for fast lookup.
- Debugging (e.g., finding the commit that introduced a bug in Git – “git bisect”).

11. Summary Table

Feature	Binary Search
Works on sorted?	Yes
Works on unsorted?	No
Time Complexity	$O(\log n)$
Space Complexity	$O(1)$ iterative, $O(\log n)$ recursive
Best Case	$O(1)$
Worst Case	$O(\log n)$

12. Practice Tasks

1. Implement recursive binary search.
2. Modify algorithm to return all occurrences of target.
3. Find first and last occurrence in a sorted array with duplicates.
4. Implement binary search in a rotated sorted array.
5. Use binary search to solve the “square root” problem.

13. Closing Thought

Binary Search is not just a searching technique—it’s a **paradigm** of algorithm design. Mastering it opens the door to solving a wide variety of computational problems efficiently.